

DevSecOps Chain B: Prove

Build Provenance, Keyless Signing and Admission Control for DocuTrust

Expadox Labs · DevSecOps Track · Projects 4, 5 and 6

Author: Kosisochukwu Eneh · Built on macOS (Apple Silicon) · Windows steps included

Contents

About this guide

- How to read the boxes
- Prerequisites

Part 0: Set up your machine

- 0.1 Windows: use WSL2
- 0.2 Install the tools
- 0.3 Cosign (two versions) and crane
- 0.4 Sign in to GitHub from the terminal
- 0.5 Set your variables once

Part 1: Project 4: SLSA Provenance and Ephemeral Build Runners

- Step 0: Fork and clone the repository
- Step 1: Baseline SLSA audit (Deliverable 1)
- Step 2: Add a provenance workflow (Deliverable 2)
- Step 3: Verify the provenance independently (Deliverable 3)
- Step 4: How the signing identity stays out of the build's hands (Deliverable 4)
- Step 5: Adversarial test: can a build step get the signing token? (Deliverable 5)
- Step 6: Prove the runners are ephemeral (Deliverable 6)
- Step 7: Cross-check with a second verifier (Deliverable 7)
- Step 8: Claim a SLSA level, with evidence (Deliverable 8)
- Step 9: Self-assessment against SLSA v1.0 (Deliverable 9)
- Step 10: Handoff to Project 5 (Deliverable 10)

Part 2: Project 5: Sigstore Keyless Signing and in-toto Attestations

- Step 1: Sign the image and attest the SBOM in CI (Deliverables 1 and 4)
- Step 2: Verify the signature (Deliverable 2)
- Step 3: Find the signature in the public log (Deliverable 3)
- Step 4: Check the SBOM contents (Deliverable 5)
- Step 5: Verify every claim on one digest (Deliverable 6)
- Step 6: Tampering test (Deliverable 7)
- Step 7: Identity forgery test (Deliverable 8)
- Step 8: Write the verification policy (Deliverable 9)
- Step 9: Handoff to Project 6 (Deliverable 10)

Part 3: Project 6: Admission-Time Enforcement and Attestation Graph Correlation

- Step 1: Create a cluster and install Kyverno (Deliverable 1)
- Step 2: Signature policy (Deliverable 2)
- Step 3: Positive test: the signed image is admitted (Deliverable 3)
- Step 4: Negative test: an unsigned image is rejected (Deliverable 4)
- Step 5: Negative test: the tampered image is rejected (Deliverable 5)
- Step 6: SBOM attestation policy (Deliverable 6)
- Step 7: Install GUAC and load the evidence (Deliverable 7)
- Step 8: Answer the Iodash question from the graph (Deliverable 8)
- Step 9: Honest scope assessment of GUAC (Deliverable 9)
- Step 10: Chain B closing report (Deliverable 10)
- Step 11: Commit your files and clean up

Appendix A: The complete `provenance.yml`

Appendix B: Error index

Appendix C: Versions used in the reference build

Acknowledgements

About this guide

Chain B answers one question about the DocuTrust container image: **can we prove where it came from, that it hasn't been changed, and stop anything we can't prove from running?**

Part	Project	What you build
1	Project 4: SLSA provenance	A GitHub Actions workflow that builds the image, pushes it to GHCR and attaches signed SLSA provenance. Then you test it: forgery attempt, ephemeral runners, two verifiers.
2	Project 5: Sigstore signing and SBOM	Keyless Cosign signing and a signed SBOM, both from CI. Then a tampering test and an identity-forgery test.
3	Project 6: Admission control and graph correlation	Kyverno on a local Kubernetes cluster that only admits images signed by your pipeline. GUAC to answer "is our image affected by a known vulnerable library?"

Every step shows the command, what you should see, and what to do if you see something else. Output blocks come from the reference build of this guide. Your digests, run IDs and timestamps will be different.

Copy commands from the companion file `DevSecOps-Chain-B-Guide.md`, not from this PDF. PDF viewers break long lines when you copy them, and a broken line fails in the terminal.

How to read the boxes

WINDOWS

Windows users: a different command or install step.

GITHUB WEBSITE

The same thing done in the GitHub website instead of the terminal.

IF YOU SEE THIS

An error you may hit, and how to fix it. Every one of these came up in the reference build.

CAPTURE

a point where you should take your own screenshot for your evidence record. Where the reference build's screenshot is included, it appears as a numbered figure instead.

NOTE

Something to understand before moving on.

Prerequisites

- A GitHub account.
- About 8 GB of free RAM for Docker and the local Kubernetes cluster.
- Basic terminal use: `cd`, copying commands, reading output.
- Projects 1–3 (Chain A: Detect) are recommended but not required. Chain B works at the build and deployment layer, not the application code.

NOTE

Chain B starts from the base DocuTrust repository, **without** the Project 1–3 code fixes applied. The seeded weaknesses, such as the SQL injection and lodash 4.17.15, are still in the image. That's deliberate: Part 3 uses the unpatched lodash as its test case. If your fork already has the Project 1–3 fixes, Part 3 will show the patched lodash version instead.

Part 0: Set up your machine

0.1 Windows: use WSL2

On Windows, do everything inside **WSL2 (Ubuntu)**. Then every command in this guide works as written.

WINDOWS

Open PowerShell **as Administrator** and run:

```
wsl --install -d Ubuntu
```

Restart when asked, open **Ubuntu** from the Start menu, and create your Linux user. From here on, "terminal" means the Ubuntu window.

Install **Docker Desktop for Windows**, then open *Settings* → *Resources* → *WSL Integration* and switch on **Ubuntu**. Run `docker version` inside Ubuntu to confirm it works.

0.2 Install the tools

Tool	Used for	macOS	Windows (inside WSL Ubuntu)
Git	Source control	<code>brew install git</code>	<code>sudo apt update && sudo apt install -y git</code>
GitHub CLI (<code>gh</code>)	Runs, attestations	<code>brew install gh</code>	Follow github.com/cli/cli/blob/trunk/docs/install_linux.md (apt section)
Docker	Local builds; runs Cosign v2.5.2 and crane	Docker Desktop for Mac	Docker Desktop + WSL integration (0.1)
jq	Reading JSON output	<code>brew install jq</code>	<code>sudo apt install -y jq</code>
Syft	SBOM generation (optional locally)	<code>brew install syft</code>	<code>curl -sSfL -o syft-install.sh https://raw.githubusercontent.com/anchore/syft/main/install.sh && sudo sh syft-install.sh -b /usr/local/bin</code>
slsa-verifier	Project 4, Step 7	<code>brew install slsa-verifier</code>	<code>sudo apt install -y golang-go && go install github.com/slsa-framework/slsa-verifier/v2/cli/slsa-verifier@latest</code> then add <code>\$(go env GOPATH)/bin</code> to PATH

Tool	Used for	macOS	Windows (inside WSL Ubuntu)
kubectl	Kubernetes	<code>brew install kubectl</code>	<code>sudo snap install kubectl --classic</code> , or the curl method on kubernetes.io/docs/tasks/tools
k3d	Local Kubernetes cluster	<code>brew install k3d</code>	<code>curl -s -o k3d-install.sh https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh && bash k3d-install.sh</code>
Helm	Installing Kyverno	<code>brew install helm</code>	<code>curl -s -o get-helm-3.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 && bash get-helm-3.sh</code>

On a Mac without Homebrew, install it first from brew.sh.

0.3 Cosign (two versions) and crane

The pipeline signs with **Cosign v2.5.2**. Cosign v3 stores and looks up signatures differently, and mixing the two causes the most confusing errors in this chain. So the guide uses two commands:

Command	Version	Use it for
<code>cosign2</code>	v2.5.2, run through Docker	Every <code>verify</code> and <code>verify-attestation</code> in this guide
<code>cosign</code>	Latest (v3), installed normally	Only <code>cosign tree</code> , which lists everything attached to an image

Install the normal `cosign` :

- **macOS:** `brew install cosign`
- **WSL:**

```
curl -sSfL -o cosign https://github.com/sigstore/cosign/releases/latest/download/cosign-linux-amd64
sudo install cosign /usr/local/bin/cosign && rm cosign
```

Then create the two Docker-based commands. These work the same on macOS and WSL:

```
alias cosign2='docker run --rm ghcr.io/sigstore/cosign/cosign:v2.5.2'
alias crane='docker run --rm gcr.io/go-containerregistry/crane:latest'
cosign version
cosign2 version
```

Expected: `cosign version` shows v3.x and `cosign2 version` shows `GitVersion: v2.5.2` . The first `cosign2` run downloads the image.

```
docutrust % cosign version

cosign: A tool for Container Signing, Verification and Storage in an OCI registry

GitVersion:      v3.1.3
GitCommit:       11926fa5bbbbde47e88fc006b625a17769b743b2
GitTreeState:    "clean"
BuildDate:       2026-08-05T23:43:27Z
GoVersion:       go1.26.5
Compiler:        gc
Platform:        darwin/arm64
```

Figure 1: cosign version on macOS, showing the locally installed Cosign v3

NOTE

An alias only lasts for the current terminal window. To keep them, add both `alias` lines to `~/.zshrc` (macOS) or `~/.bashrc` (WSL), then open a new terminal.

IF YOU SEE THIS

none of the attestations matched the predicate type: spdxjson, found:
<https://slsa.dev/provenance/v1> . You verified a v2.5.2 SBOM attestation with Cosign v3. Use `cosign2` .

0.4 Sign in to GitHub from the terminal

```
gh auth login
```

Choose **GitHub.com** → **HTTPS** → **Login with a web browser**. Copy the one-time code, paste it in the browser page that opens, and approve.

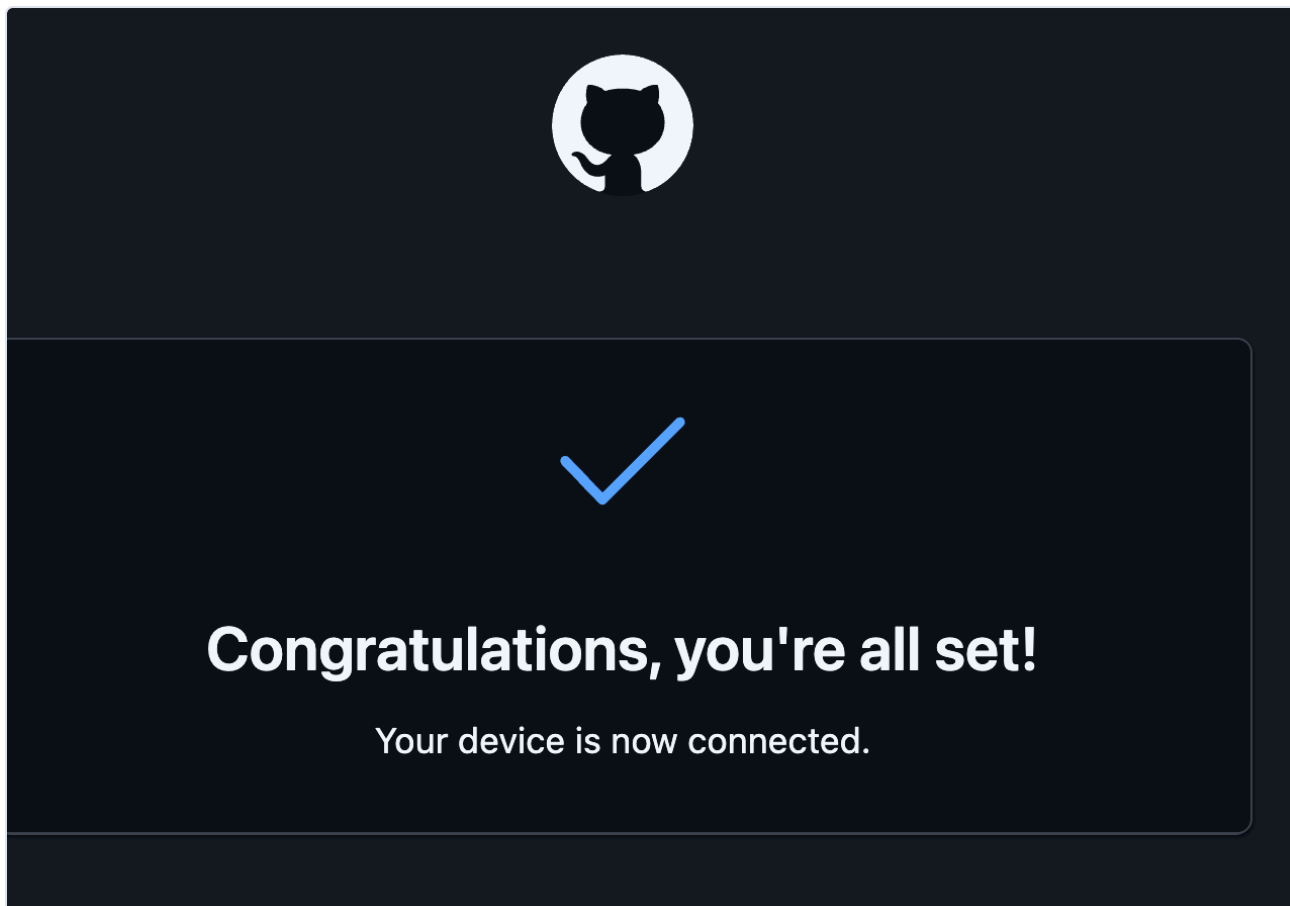


Figure 2: the browser page confirming the GitHub CLI device is authorized.

```
gh auth status
```

Expected: Logged in to github.com account <your-username> .

0.5 Set your variables once

Many commands below use these. Set them in every new terminal, or add them to your shell profile.

```
export GH_USER=<your-github-username>
export OWNER=$(echo "$GH_USER" | tr '[:upper:]' '[:lower:]')
export IMAGE=ghcr.io/$OWNER/docutrust
echo $IMAGE
```

GHCR image names must be **lowercase**, which is why `OWNER` exists. In the reference build: `GH_USER=KoceeEneh` and `IMAGE=ghcr.io/koceeeneh/docutrust` .

Part 1: Project 4: SLSA Provenance and Ephemeral Build Runners

Goal: prove where every DocuTrust image came from. You will add signed SLSA provenance to CI, verify it, try to break it, and score it honestly against SLSA v1.0.

Step 0: Fork and clone the repository

0.1 Fork. Open github.com/expadox/docustrust and click **Fork** (top right) → **Create fork**.

GITHUB WEBSITE

That's the only way to fork. There's no terminal step for it. `gh repo fork expadox/docustrust --clone` does the fork and the clone in one command, if you prefer.

0.2 Clone your fork.

```
git clone https://github.com/$GH_USER/docustrust.git
cd docustrust
git remote -v
```

Expected: both lines show **your** username, not `expadox`.

```
origin https://github.com/KoceeEneh/docustrust.git (fetch)
origin https://github.com/KoceeEneh/docustrust.git (push)
```

IF YOU SEE THIS

If `origin` shows `expadox`, you cloned the original. Every push will fail. Delete the folder and clone your fork.

0.3 Point `gh` at your fork. If you have other repos, `gh` can guess the wrong one.

```
gh repo set-default $GH_USER/docustrust
```

Run every command from here on inside the `docustrust` folder, unless a step says otherwise.

Step 1: Baseline SLSA audit (Deliverable 1)

Record the starting state before you change anything.

```
cat .github/workflows/ci.yml
```

What you'll see: one job, `build-and-test`. It runs `npm ci` and `docker build -t docustrust:ci-${{ github.sha }}`, followed by a comment block saying later projects add their own stages.

Check whether the code fixes from Projects 1–3 are present:

```
grep -n "SELECT" src/routes/documents.js
```

```
37:   const result = await pool.query("SELECT * FROM documents WHERE id = $1", [req.params.id]);
76:   const query = `SELECT id, title FROM documents WHERE title ILIKE '${searchTerm}%'`;
93:   const result = await pool.query("SELECT title, body FROM documents WHERE id = $1", [
```

Line 76 puts user input straight into SQL: the seeded SQL injection is still there.

Write the audit. Copy this table into your report:

SLSA v1.0 Build Track requirement	Met?
Provenance is generated	No: there's no provenance step
Provenance is signed	No: nothing is signed
Hosted build platform	Partly: GitHub-hosted runners, but that earns no level without provenance
Image published to a registry	No: the image is built on the runner and thrown away

Result: SLSA Build Level 0.

CAPTURE the ci.yml output and the grep output in your terminal.

Step 2: Add a provenance workflow (Deliverable 2)

You'll add a **separate** workflow, `provenance.yml`. It runs only on pushes to `main`, builds the image, pushes it to GHCR and signs provenance for that exact image. `ci.yml` stays as the fast check on pull requests.

2.1 Create the file.

```
mkdir -p .github/workflows
nano .github/workflows/provenance.yml
```

Paste the following, then save with **Ctrl+O**, **Enter** and exit with **Ctrl+X**.

```
name: Build, Publish, and Attest Provenance

on:
  push:
    branches: [main]

permissions:
  contents: read
  packages: write      # push the image to GHCR
  id-token: write      # request an OIDC token for keyless signing
  attestations: write  # store the attestation in the repo

env:
  REGISTRY: ghcr.io

jobs:
  build-publish-attest:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Compute lowercase image name
        id: image
        run: |
          echo "name=$(echo '${{ github.repository }}' | tr '[:upper:]' '[:lower:]')" >> "$GITHUB_OUTPUT"

      - name: Log in to GHCR
        uses: docker/login-action@v3
        with:
          registry: ${ env.REGISTRY }
          username: ${ github.actor }
```

```

    password: ${ secrets.GITHUB_TOKEN }}

- name: Build and push image
  id: push
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: ${ env.REGISTRY }}/${ steps.image.outputs.name }}:${ github.sha }}

- name: Generate build provenance attestation
  uses: actions/attest-build-provenance@v2
  with:
    subject-name: ${ env.REGISTRY }}/${ steps.image.outputs.name }}
    subject-digest: ${ steps.push.outputs.digest }}
    push-to-registry: true

```

What each part does:

Part	Why it's there
<code>id-token: write</code>	Lets the job ask GitHub for a short-lived identity token. Sigstore turns that into a ~10-minute signing certificate. No signing key is created or stored anywhere.
<code>attestations: write</code>	Lets the job save the attestation to your repo.
<code>packages: write</code>	Lets the job push to GHCR. The built-in <code>GITHUB_TOKEN</code> is enough, so you don't create any secret.
<code>Compute lowercase image name</code>	Registry names must be lowercase. <code>github.repository</code> keeps your capitals.
<code>subject-digest</code>	The provenance is tied to the image's content hash , not a tag. Anyone can move a tag; they can't change a digest.
<code>push-to-registry: true</code>	Stores the attestation next to the image in GHCR, as well as in GitHub.

GITHUB WEBSITE

You can create the file on GitHub instead: in your fork, click **Add file** → **Create new file**, name it `.github/workflows/provenance.yml`, paste the YAML and click **Commit changes** on `main`. Then run `git pull` locally.

IF YOU SEE THIS

ERROR: failed to build: invalid tag "ghcr.io/KoceeEneh/docutrust:...": repository name must be lowercase. This happens if you use `${ github.repository }` directly in `tags`. The `Compute lowercase image name` step above prevents it.

2.2 Commit and push.

```

git add .github/workflows/provenance.yml
git commit -m "Project 4: build, publish and attest provenance"
git push

```

2.3 Watch the run.

```
gh run list --workflow=provenance.yml --limit 3
```

STATUS	TITLE	WORKFLOW	BRANCH	EVENT	ID
✓	Project 4: bui...	Build, Publish, and Attest Prove...	main	push	32426565622

The ID column is often cut off. Get the latest run ID on its own and save it:

```
RUN_ID=$(gh run list --workflow=provenance.yml --limit 1 --json databaseId --jq '[0].databaseId')
echo $RUN_ID
gh run watch $RUN_ID
```

GITHUB WEBSITE

Open your fork → **Actions** tab → **Build, Publish, and Attest Provenance** → the latest run. Click a step to read its log.

2.4 Pull the evidence from the log.

```
gh run view $RUN_ID --log | grep -E "containerimage.digest|Attestation created|logIndex|attestations/"
```

Example output (trimmed):

```
containerimage.digest: sha256:3ec8a2bbe9124ed9c1af52c381062080251e2a83baf87bd4306a8a69f90a7a5d
Attestation created for ghcr.io/koceeeneh/docutrust@sha256:3ec8a2bb...
https://search.sigstore.dev?logIndex=2539629983
https://github.com/KoceeEneh/docutrust/attestations/42000529
```

Save the digest for the next steps:

```
export DIGEST=$(gh run view $RUN_ID --log | grep -o '"containerimage.digest": "sha256:[a-f0-9]*"' | gr
ep -o 'sha256:[a-f0-9]*' | head -1)
echo $DIGEST
```

2.5 Make the package public. The local cluster in Part 3 pulls this image without credentials, and anyone replicating your checks needs to read it.

GITHUB WEBSITE

GitHub → your profile → **Packages** → **docutrust** → **Package settings** → **Danger Zone** → **Change visibility** → **Public**.

CAPTURE the green run in the Actions tab, with the Generate build provenance attestation step expanded.

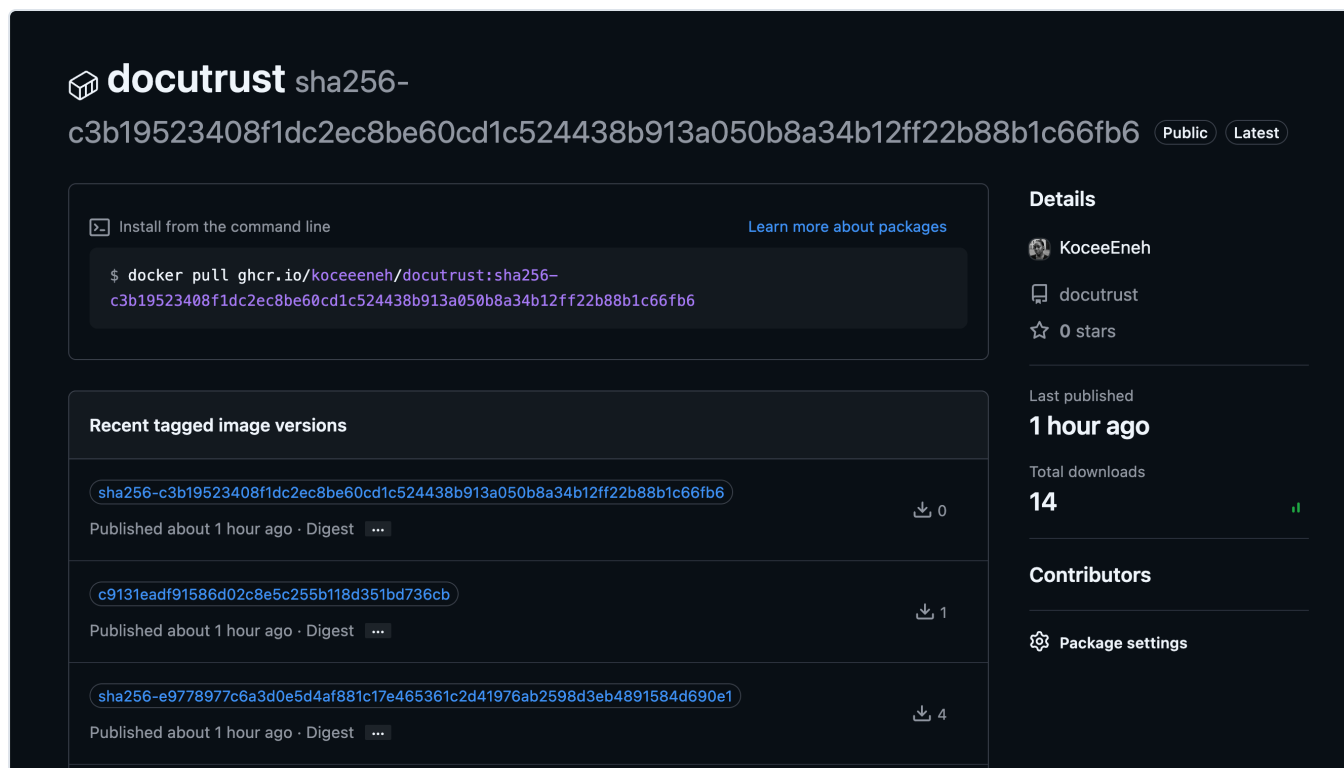


Figure 3: the package page on GitHub showing the pushed image.

Step 3: Verify the provenance independently (Deliverable 3)

A log saying "attestation created" isn't proof. Verify it with a separate tool:

```
gh attestation verify oci://$IMAGE@$DIGEST --owner $GH_USER
```

```
Loaded 1 attestation from GitHub API
The following policy criteria will be enforced:
- Predicate type must match:..... https://slsa.dev/provenance/v1
- Source Repository Owner URI must match:... https://github.com/KoceeEneh
- Subject Alternative Name must match regex: (?i)^https://github\.com/KoceeEneh/
- OIDC Issuer must match:..... https://token.actions.githubusercontent.com
✓ Verification succeeded!

- Attestation #1
- Build repo:..... KoceeEneh/docutrust
- Build workflow:.. .github/workflows/provenance.yml@refs/heads/main
- Signer repo:.... KoceeEneh/docutrust
- Signer workflow: .github/workflows/provenance.yml@refs/heads/main
```

What `gh` checked: the attestation is SLSA v1 provenance, it was signed by an identity from **your** repos, and the token came from GitHub's real OIDC issuer. It also names the exact workflow file that built the image.

To see the provenance content itself:

```
gh attestation verify oci://$IMAGE@$DIGEST --owner $GH_USER --format json \
| jq '.[0].verificationResult.statement.predicate.buildDefinition.resolvedDependencies'
```

It shows the git commit the image was built from. That's the link from image back to source.

GITHUB WEBSITE

Open the <https://github.com/<you>/docutrust/attestations/<id>> link from Step 2.4 to see the same attestation in the browser.

```
docutrust % gh attestation verify oci://ghcr.io/kocee
neh/docutrust@sha256:14c424be8b733f0eadb241745bbded59db441099efd8d5e06f79412a0a33f7a3 --ow
ner KoceeEneh
Loaded digest sha256:14c424be8b733f0eadb241745bbded59db441099efd8d5e06f79412a0a33f7a3 for
oci://ghcr.io/koceeeneh/docutrust@sha256:14c424be8b733f0eadb241745bbded59db441099efd8d5e06
f79412a0a33f7a3
Loaded 1 attestation from GitHub API

The following policy criteria will be enforced:
- Predicate type must match:..... https://slsa.dev/provenance/v1
- Source Repository Owner URI must match:... https://github.com/KoceeEneh
- Subject Alternative Name must match regex: (?i)^https://github\.com/KoceeEneh/
- OIDC Issuer must match:..... https://token.actions.githubusercontent.com

✓ Verification succeeded!

The following 1 attestation matched the policy criteria

- Attestation #1
  - Build repo:..... KoceeEneh/docutrust
  - Build workflow:. .github/workflows/provenance.yml@refs/heads/main
  - Signer repo:.... KoceeEneh/docutrust
  - Signer workflow: .github/workflows/provenance.yml@refs/heads/main
```

Figure 4: ✓ Verification succeeded! with the build and signer workflow lines.

Step 4: How the signing identity stays out of the build's hands (Deliverable 4)

There's no command here. You'll read your own evidence and write this section of the report.

Stage	Who controls it	Where to see it
The job asks for a token	The job, only because of <code>id-token: write</code>	<code>provenance.yml</code>
The token's claims (repo, workflow, branch)	GitHub , not the job	Step 5 decodes a real token
The ~10-minute signing certificate	Sigstore Fulcio	Open <a href="https://search.sigstore.dev?logIndex=<your index>">https://search.sigstore.dev?logIndex=<your index> from Step 2.4 and read the certificate's validity dates
The permanent public log entry	Sigstore Rekor	The same search.sigstore.dev page
The check at verify time	<code>gh attestation verify</code>	Step 3 output

What to write: no long-lived key exists anywhere. The build steps can **ask** for a token, but they can't choose what identity it carries. Step 5 tests exactly where that stops.

CAPTURE the search.sigstore.dev entry showing the certificate's validity window.

Step 5: Adversarial test: can a build step get the signing token? (Deliverable 5)

`id-token: write` applies to the **whole job**. So any step, including a malicious dependency's install script, may be able to request the same token. Test it on a throwaway branch.

5.1 Create a test branch.

```
git checkout -b project4-adversarial-test
```

5.2 Let the workflow run manually. Push-only workflows can't be triggered by hand. In `provenance.yml`, change the `on:` block to:

```
on:
  push:
    branches: [main]
  workflow_dispatch:
```

IF YOU SEE THIS

HTTP 422: Workflow does not have 'workflow_dispatch' trigger. You tried `gh workflow run` before adding `workflow_dispatch: .`

5.3 Add the attacker step immediately **after** `Checkout`, so it runs before the real signing step:

```
- name: ADVERSARIAL TEST: attempt to read and decode the OIDC identity token
  run: |
    echo "Attempting to fetch the OIDC ID token available to this build step..."
    TOKEN=$(curl -sS -H "Authorization: bearer $ACTIONS_ID_TOKEN_REQUEST_TOKEN" \
      "$ACTIONS_ID_TOKEN_REQUEST_URL&audience=sigstore" | jq -r '.value')
    echo "Token acquired: $([ -n "$TOKEN" ] && echo YES || echo NO)"
    echo "--- Decoded JWT payload (claims) ---"
    echo "$TOKEN" | cut -d. -f2 | base64 -d 2>/dev/null | jq .
```

The step prints the token's **claims** (who it says it is), never the token itself.

5.4 Push the branch and run it.

```
git add .github/workflows/provenance.yml
git commit -m "Project 4: adversarial OIDC token test (test branch only)"
git push -u origin project4-adversarial-test
gh workflow run provenance.yml --ref project4-adversarial-test
```

GITHUB WEBSITE

Actions → **Build, Publish, and Attest Provenance** → **Run workflow** → choose branch `project4-adversarial-test` → **Run workflow**.

5.5 Read the result.

```
RUN_ID=$(gh run list --workflow=provenance.yml --branch project4-adversarial-test --limit 1 --json databaseId --jq '[0].databaseId')
gh run watch $RUN_ID
gh run view $RUN_ID --log | grep -A 30 "ADVERSARIAL TEST"
```

```
Token acquired: YES
--- Decoded JWT payload (claims) ---
{
```

```

"actor": "KoceeEneh",
"aud": "sigstore",
"event_name": "workflow_dispatch",
"iss": "https://token.actions.githubusercontent.com",
"job_workflow_ref": "KoceeEneh/docutrust/.github/workflows/provenance.yml@refs/heads/project4-adversarial-test",
"repository": "KoceeEneh/docutrust",
"sub": "repo:KoceeEneh@<account-id>/docutrust@<repo-id>:ref:refs/heads/project4-adversarial-test",
...
}

```

What this shows (write it up plainly):

- **The build step got a real signing token.** Signing isn't isolated from the build steps. They share the same job.
- **The step could not change the identity.** GitHub sets every claim from the real run: repo, workflow, branch. A forged signature would still say exactly where it came from.
- **Damage is limited** by the ~10-minute certificate lifetime and the public Rekor log entry that every signing creates.

This is the one SLSA Level 3 requirement the pipeline doesn't meet (Step 8).

5.6 Clean up. Go back to `main`. The test branch stays on GitHub as evidence; delete it if you don't want it.

```

git checkout main
# optional: git push origin --delete project4-adversarial-test

```

CAPTURE Token acquired: YES and the decoded claims in the run log.

Step 6: Prove the runners are ephemeral (Deliverable 6)

Leave a file on the runner in one run, then check for it in a later run.

6.1 Run 1: plant a marker. On `main`, add this step right after `Checkout` in `provenance.yml`:

```

- name: EPHEMERAL TEST: write a marker file and report disk state
  run: |
    echo "planted-by-run-`${github.run_id}`-at-$(date -u +%s)" > /tmp/docutrust-ephemeral-marker.txt
    cat /tmp/docutrust-ephemeral-marker.txt
    echo "Runner hostname: $(hostname)"
    echo "Runner machine ID (if available):"
    cat /etc/machine-id 2>/dev/null || echo "no /etc/machine-id"

```

```

git add .github/workflows/provenance.yml
git commit -m "Project 4: ephemeral runner test (run 1)"
git push
RUN_ID=$(gh run list --workflow=provenance.yml --limit 1 --json databaseId --jq '[0].databaseId')
gh run watch $RUN_ID
gh run view $RUN_ID --log | grep -A 6 "EPHEMERAL TEST"

```

```

planted-by-run-32535889744-at-1787353836
Runner hostname: runnervm76f27
Runner machine ID (if available):
1807f263ef384af2a1c72845a603b329

```

6.2 Run 2: look for it. Replace the whole step with this version, then push:

```

- name: EPHEMERAL TEST: check for prior marker, then plant a new one
  run: |

```

```

if [ -f /tmp/docutrust-ephemeral-marker.txt ]; then
    echo "MARKER FOUND (unexpected): $(cat /tmp/docutrust-ephemeral-marker.txt)"
else
    echo "MARKER NOT FOUND: no file at /tmp/docutrust-ephemeral-marker.txt"
fi
echo "Runner hostname: $(hostname)"
echo "Runner machine ID (if available):"
cat /etc/machine-id 2>/dev/null || echo "no /etc/machine-id"
echo "planted-by-run-{{ github.run_id }}-at-$(date -u +%s)" > /tmp/docutrust-ephemeral-mark
er.txt

```

```

git add .github/workflows/provenance.yml
git commit -m "Project 4: ephemeral runner test (run 2)"
git push
RUN_ID=$(gh run list --workflow=provenance.yml --limit 1 --json databaseId --jq '[0].databaseId')
gh run watch $RUN_ID
gh run view $RUN_ID --log | grep -A 6 "EPHEMERAL TEST"

```

```

MARKER NOT FOUND: no file at /tmp/docutrust-ephemeral-marker.txt
Runner hostname: runnervm76f27
Runner machine ID (if available):
1807f263ef384af2a1c72845a603b329

```

Result: the file from Run 1 is gone. The runners don't keep state between runs.

NOTE

The hostname and machine ID can be **identical** in both runs, as in mine. That doesn't mean the machine was reused. GitHub builds its runners from a shared VM image, and those IDs can be baked into it. The file test is the reliable signal.

6.3 Remove the test step. Delete the `EPHEMERAL TEST` step from `provenance.yml`, then commit and push.

```

git add .github/workflows/provenance.yml
git commit -m "Project 4: remove ephemeral runner test step"
git push

```

CAPTURE Run 1's marker output next to Run 2's MARKER NOT FOUND.

Step 7: Cross-check with a second verifier (Deliverable 7)

Refresh `DIGEST` to the newest image, then try `slsa-verifier`:

```

RUN_ID=$(gh run list --workflow=provenance.yml --limit 1 --json databaseId --jq '[0].databaseId')
export DIGEST=$(gh run view $RUN_ID --log | grep -o '"containerimage.digest": "sha256:[a-f0-9]*"' | gr
ep -o 'sha256:[a-f0-9]*' | head -1)
slsa-verifier verify-image $IMAGE@$DIGEST --source-uri github.com/$GH_USER/docutrust

```

```

FAILED: SLSA verification failed: no matching attestations:

```

Check that the attestation really exists before drawing a conclusion:

```

cosign tree $IMAGE@$DIGEST

```

 Supply Chain Security Related artifacts for an image: `ghcr.io/koceeeneh/docutrust@sha256:e9778977...`

```
└─ https://slsa.dev/provenance/v1 artifacts via OCI referrer: ghcr.io/koceeeneh/docustrust@sha256:c
bfb800c...
└─ sha256:3d806363...
```

The provenance is there. `slsa-verifier` was built for the older `slsa-github-generator` format. Its own README recommends `gh attestation verify` for attestations made with GitHub's `attest-build-provenance`.

	gh attestation verify	slsa-verifier
Made by	GitHub	The SLSA project
Built for	GitHub artifact attestations	slsa-github-generator output
Result on this image	✓ Verified	✗ no matching attestations

NOTE

This is an expected result, not a failure on your part. The takeaway for the report: don't assume every "SLSA" tool reads every SLSA attestation. Test it.

```
❌ docustrust % slsa-verifier verify-image g
hcr.io/koceeeneh/docustrust@sha256:e9778977c6a3d0e5d4af881c17e465361c
2d41976ab2598d3eb4891584d690e1 \
--source-uri github.com/KoceeEneh/docustrust
FAILED: SLSA verification failed: no matching attestations:
/opt/homebrew/share/zsh/site-functions
● docustrust % cosign tree ghcr.io/koceeene
h/docustrust@sha256:e9778977c6a3d0e5d4af881c17e465361c2d41976ab2598d3
eb4891584d690e1
📦 Supply Chain Security Related artifacts for an image: ghcr.io/koc
eeeneh/docustrust@sha256:e9778977c6a3d0e5d4af881c17e465361c2d41976ab2
598d3eb4891584d690e1
└─ https://slsa.dev/provenance/v1 artifacts via OCI referrer: gh
cr.io/koceeeneh/docustrust@sha256:cbfb800c63b78ddde30cf01613b1e2aa4cd
895abb026e89c6974d8693999f5e2
└─ sha256:3d806363367ed9c353061bca8728105b799e2ac023fc41febd7
368f2322378cc
● docustrust % cosign tree ghcr.io/koceeene
h/docustrust@sha256:e9778977c6a3d0e5d4af881c17e465361c2d41976ab2598d3
eb4891584d690e1
📦 Supply Chain Security Related artifacts for an image: ghcr.io/koc
eeeneh/docustrust@sha256:e9778977c6a3d0e5d4af881c17e465361c2d41976ab2
598d3eb4891584d690e1
└─ https://slsa.dev/provenance/v1 artifacts via OCI referrer: gh
cr.io/koceeeneh/docustrust@sha256:cbfb800c63b78ddde30cf01613b1e2aa4cd
895abb026e89c6974d8693999f5e2
└─ sha256:3d806363367ed9c353061bca8728105b799e2ac023fc41febd7
368f2322378cc
```

Figure 5: the `slsa-verifier` failure and the `cosign tree` output together.

Step 8: Claim a SLSA level, with evidence (Deliverable 8)

Level	Requirement	Evidence	Met?
L1	Provenance exists	Step 2	Yes
L2	Signed provenance, hosted build platform	Step 3, GitHub-hosted runners	Yes

Level	Requirement	Evidence	Met?
L3	Identity can't be forged	Step 5: claims set by GitHub	Yes
L3	Ephemeral build environment	Step 6	Yes
L3	Build steps can't reach the signing process	Step 5: they can get the token	No

Claim: SLSA Build Level 2, with every Level 3 requirement met except isolation between build steps and signing.

Step 9: Self-assessment against SLSA v1.0 (Deliverable 9)

Track / requirement	Status
Build L1 / L2	Met
Build L3: unforgeable identity, ephemeral environment	Met
Build L3: signing isolated from build steps	Not met: tested in Step 5
Provenance fields (<code>builder.id</code> , <code>buildType</code> , <code>externalParameters</code> , <code>resolvedDependencies</code> , <code>run metadata</code>)	Present: check with the <code>jq</code> command in Step 3
Source Track (branch protection, required review)	Not attempted

NOTE

Optional extension, the Source Track. It needs a second person to review pull requests. Add them under **Settings** → **Collaborators**, then protect `main` under **Settings** → **Branches** → **Add branch protection rule** → **Require a pull request before merging** → **Require approvals: 1**. Test it: a PR merged without approval must be blocked. The reference build doesn't cover this step.

Step 10: Handoff to Project 5 (Deliverable 10)

Write a short handoff with:

- **Repo, registry and workflow:** `github.com/<you>/docutrust` , `ghcr.io/<you-lowercase>/docutrust` , `.github/workflows/provenance.yml` on push to `main` .
- **How to verify:** `gh attestation verify oci://<image>@<digest> --owner <you>` .
- **Known gap:** build steps can request the signing token (Step 5). Project 5's signing step runs in the same job and has the same gap.
- **Tool note:** use `gh attestation verify` , not `slsa-verifier` , for this pipeline (Step 7).

Part 2: Project 5: Sigstore Keyless Signing and in-toto Attestations

Goal: prove the image itself and its contents are genuine. You'll add a keyless Cosign signature and a signed SBOM to the pipeline, verify both, then try to fool the verification with a tampered image and a wrong identity.

Step 1: Sign the image and attest the SBOM in CI (Deliverables 1 and 4)

You'll add both to the pipeline in one change: the Cosign signature (Deliverable 1) and the SBOM attestation (Deliverable 4). Signing and attesting in CI means the pipeline's identity signs everything, and no person's.

1.1 Open `.github/workflows/provenance.yml` and add these steps at the **end** of `steps:`, after `Generate build provenance attestation`:

```
- name: Install cosign
  uses: sigstore/cosign-installer@v3
  with:
    cosign-release: 'v2.5.2'

- name: Sign the image with Cosign (keyless)
  run: |
    cosign sign --yes ${env.REGISTRY}/${steps.image.outputs.name}@${steps.push.output
s.digest }}

- name: Record cosign version
  run: cosign version

- name: Install Syft
  uses: anchore/sbom-action/download-syft@v0
  with:
    syft-version: v1.42.3

- name: Generate SBOM from the pushed image
  run: |
    syft ${env.REGISTRY}/${steps.image.outputs.name}@${steps.push.outputs.digest }} \
      -o spdx-json > sbom.json
    echo "Packages found: $(jq '.packages | length' sbom.json)"

- name: Attest SBOM with Cosign (workflow identity)
  run: |
    cosign attest --yes \
      --predicate sbom.json \
      --type spdxjson \
      ${env.REGISTRY}/${steps.image.outputs.name}@${steps.push.outputs.digest }}
```

Step	What it does
<code>cosign sign --yes</code>	Signs the image digest with a short-lived certificate for the workflow's identity. <code>--yes</code> skips the confirmation prompt, since nobody is there to answer it in CI.
<code>syft ... -o spdx-json</code>	Lists every package inside the pushed image, in the SPDX format.
<code>cosign attest --type spdxjson</code>	Wraps the SBOM in a signed in-toto statement tied to the image digest.
<code>cosign-release / syft-version</code>	Pin the versions. Without the pin, CI picks up whatever is newest, and Syft changes the identifiers it writes between versions (Part 3, Step 8).

```

- name: Install cosign
  uses: sigstore/cosign-installer@v3

- name: Sign the image with Cosign (keyless)
  run: |
    cosign sign --yes ${env.REGISTRY}/${steps.image.out$

```

Figure 6: The Cosign steps added at the end of provenance.yml

IF YOU SEE THIS

Error: signing [ghcr.io/@]: parsing reference: could not parse reference: ghcr.io/@. The signing step runs **before** the build step, so `steps.push.outputs.digest` is still empty. Move the Cosign steps to the end of the file.

1.2 Push and watch the run.

```

git add .github/workflows/provenance.yml
git commit -m "Project 5: keyless Cosign signing and SBOM attestation in CI"
git push
RUN_ID=$(gh run list --workflow=provenance.yml --limit 1 --json databaseId --jq '.[0].databaseId')
gh run watch $RUN_ID

```

1.3 Pull the evidence and save the new digest.

```

gh run view $RUN_ID --log | grep -E "GitVersion|Packages found|tlog entry created"
export DIGEST=$(gh run view $RUN_ID --log | grep -o '"containerimage.digest": "sha256:[a-f0-9]*"' | gr
ep -o 'sha256:[a-f0-9]*' | head -1)
echo $DIGEST

```

Sign the image with Cosign (keyless)	tlog entry created with index: 2892575467
Record cosign version	GitVersion: v2.5.2
Generate SBOM from the pushed image	Packages found: 385
Attest SBOM with Cosign (workflow identity)	tlog entry created with index: 2892576479

Two `tlog entry` lines means two separate public log entries: one for the signature, one for the SBOM attestation.

NOTE

Use this `$DIGEST` for the rest of the guide. Mine was

sha256:c9167a9d3425a8f19884ca5b696d228ce6afc65dcd688a18d4573f85eaf09017 . Write yours down; a new terminal loses the variable.

CAPTURE the run log showing the two tlog entry created lines and Packages found.

Step 2: Verify the signature (Deliverable 2)

The identity is the full path to your workflow file on `main` . Type your username with the **same capitals** as on GitHub.

```

export SIGNER="https://github.com/$GH_USER/docutrust/.github/workflows/provenance.yml@refs/heads/main"
export ISSUER="https://token.actions.githubusercontent.com"

```

```
cosign2 verify \
  --certificate-identity "$SIGNER" \
  --certificate-oidc-issuer "$ISSUER" \
  $IMAGE@$DIGEST | jq '.[0].critical'
```

On screen:

The following checks were performed on each of these signatures:

- The cosign claims were validated
- Existence of the claims in the transparency log was verified offline
- The code-signing certificate was verified using trusted certificate authority certificates

The `jq` output shows `docker-manifest-digest` matching your `$DIGEST`.

NOTE

Use `--certificate-identity` (exact match), not `--certificate-identity-regexp`. The exact form trusts one workflow file on one branch. Step 9 explains why.

```
Verification for ghcr.io/koceeeneh/docutrust@sha256:14c424be8b733f0eadb241745bbded59db441099efd8d5e06f79412a0a33f7a3 --
The following checks were performed on each of these signatures:
  - The cosign claims were validated
  - Existence of the claims in the transparency log was verified offline
  - The code-signing certificate was verified using trusted certificate authority certificates

[{"critical":{"identity":{"docker-reference":"ghcr.io/koceeeneh/docutrust@sha256:14c424be8b733f0eadb241745bbded59db441099efd8d5e06f79412a0a33f7a3"},"image":{"docker-manifest-digest":"sha256:14c424be8b733f0eadb241745bbded59db441099efd8d5e06f79412a0a33f7a3"},"type":"https://slsa.dev/provenance/v1"},"optional":{}}]
```

Figure 7: the three checks, with the digest in the jq output.

Step 3: Find the signature in the public log (Deliverable 3)

Take the index from the `Sign the image with Cosign` line in Step 1.3 and open it in a browser:

```
https://search.sigstore.dev/?logIndex=<index>
```

The entry shows the signing certificate. Check that its subject is your `provenance.yml@refs/heads/main` and its issuer is `token.actions.githubusercontent.com`, and read the ~10-minute validity window.

Then list everything attached to the image:

```
cosign tree $IMAGE@$DIGEST
```

You should see three things for this one digest: the **signature**, the **SBOM attestation** (`spdx.dev/Document`) and the **SLSA provenance** (`slsa.dev/provenance/v1`).

CAPTURE the search.sigstore.dev entry for your signature.

```

docutrust % cosign tree ghcr.io/koceeeneh/docutrust@sha256:14c424be8b733f0eadb241745bbded59db441099efd8d5e06f79412a0a33f7a3
Supply Chain Security Related artifacts for an image: ghcr.io/koceeeneh/docutrust@sha256:14c424be8b733f0eadb241745bbded59db441099efd8d5e06f79412a0a33f7a3
├── 📄 Signatures for an image tag: ghcr.io/koceeeneh/docutrust:sha256-14c424be8b733f0eadb241745bbded59db441099efd8d5e06f79412a0a33f7a3.sig
├── 🍷 sha256:4009beef85cf1faec5c1fcdc8d7a6b77c8611ac104973e322f57fe5fc145e547
├── 🔗 https://spdx.dev/Document artifacts via OCI referrer: ghcr.io/koceeeneh/docutrust@sha256:3abcebf82aa0d89a70ff3108291be58ed9ca148e48d050d59f902bbc438641a0
├── 🍷 sha256:a0bc5d94289a35ce5aa272fc06c6c03875e70f40acf1486996f5dc88bb4bd7b4
├── 🔗 https://spdx.dev/Document artifacts via OCI referrer: ghcr.io/koceeeneh/docutrust@sha256:8c8b0d16dedb2875e5655c983635a8909bf11a18ad03cfb693a8c5bea2a51b7e
├── 🍷 sha256:364753183c89e99dbef40bb692e68afd340410f68294019835abace8ba0dbc14
├── 🔗 https://spdx.dev/Document artifacts via OCI referrer: ghcr.io/koceeeneh/docutrust@sha256:94f07c6608cbe42971899f03db1d6d7dfbcfb8b3be06f5bd1c1d13d2fea54f1b
├── 🍷 sha256:8063e0f265390bab2dbb7c0bcfe6a12c531a28e1af9211a2a0871be1246bda15
├── 🔗 https://slsa.dev/provenance/v1 artifacts via OCI referrer: ghcr.io/koceeeneh/docutrust@sha256:ad5f3076d73c94f9116413d5432679c2b9d7a46c9ab3d7dadbaa9b20674ce95d
├── 🍷 sha256:6b10780d3bbfe8f383312ccd8a1f5cc816146f59db2f87d5b8f37ea3270f2dfc

```

Figure 8: the cosign tree output.

Step 4: Check the SBOM contents (Deliverable 5)

A valid signature proves who signed the SBOM, not what's in it. Decode the attestation and count the packages:

```

cosign2 verify-attestation --type spdxjson \
  --certificate-identity "$SIGNER" \
  --certificate-oidc-issuer "$ISSUER" \
  $IMAGE@$DIGEST > verified-att.jsonl

head -1 verified-att.jsonl | jq -r '.payload' | base64 -d | jq '.predicate.packages | length'

```

```
385
```

That number must match `Packages found:` in the CI log. It does: the signed SBOM holds exactly what Syft found.

Save the SBOM for Part 3:

```

head -1 verified-att.jsonl | jq -r '.payload' | base64 -d | jq '.predicate' > sbom-verified.spdx.json
jq -r '.packages[] | select(.name=="lodash") | "\(.name) \(.versionInfo)"' sbom-verified.spdx.json

```

```
lodash 4.17.15
```

Keep that lodash version in mind for Part 3.

IF YOU SEE THIS

none of the attestations matched the predicate type: spdxjson. You used `cosign` (v3). Use `cosign2`.

```

docutrust % cosign verify-attestation \
--certificate-identity "132656084+KocseeEneh@users.noreply.github.com" \
--certificate-oidc-issuer "https://github.com/login/oauth" \
--type spdxjson \
ghcr.io/kocseeeneh/docutrust@sha256:14c424be8b733f0eadb241745bbded59db441099efd8d5e06f794
12a0a33f7a3 \
| jq -r '.payload' | base64 -d | jq '.predicate.packages | length'

Verification for ghcr.io/kocseeeneh/docutrust@sha256:14c424be8b733f0eadb241745bbded59db4410
99efd8d5e06f79412a0a33f7a3 --
The following checks were performed on each of these signatures:
- The cosign claims were validated
- Existence of the claims in the transparency log was verified offline
- The code-signing certificate was verified using trusted certificate authority certific
ates
385
385
385

```

Figure 9: the verification checks, followed by 385.

Step 5: Verify every claim on one digest (Deliverable 6)

One image, three independent signed claims, each checked with its own tool:

```

gh attestation verify oci://$IMAGE@$DIGEST --owner $GH_USER # prov
enance
cosign2 verify --certificate-identity "$SIGNER" --certificate-oidc-issuer "$ISSUER" $IMAGE@$DIGEST > /
dev/null # signature
cosign2 verify-attestation --type spdxjson --certificate-identity "$SIGNER" --certificate-oidc-issuer
"$ISSUER" $IMAGE@$DIGEST > /dev/null # SBOM

```

All three should succeed. Each one checks a different fact: how the image was built, that it's this exact image, and what's inside it.

CAPTURE all three verifications succeeding.

Step 6: Tampering test (Deliverable 7)

Change one line, rebuild and push under a tag that looks official. Verification must reject it.

6.1 Create a token for pushing from your machine. CI uses its built-in token, but your laptop needs its own.

GITHUB WEBSITE

GitHub → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token (classic)**. Tick **write:packages** (this includes **read:packages**), set a short expiry, generate, and copy the token.

```
docker login ghcr.io -u $GH_USER
```

Paste the token when asked for a password. Expected: **Login Succeeded**.

IF YOU SEE THIS

DENIED: permission_denied: The token provided does not match expected scopes. The token is fine-grained, or it's missing **write:packages**. Create a **classic** token with **write:packages**, run **docker logout ghcr.io**, then log in again.

IF YOU SEE THIS

denied or unauthenticated on a new computer. Registry logins are stored per machine. Run `docker login ghcr.io` on each machine you push from.

6.2 Build the tampered image.

```
echo "// tampering test marker" >> src/index.js
docker build -t $IMAGE:tampered-test .
docker push $IMAGE:tampered-test
git checkout -- src/index.js
export TAMPERED=$(docker inspect --format='{{index .RepoDigests 0}}' $IMAGE:tampered-test)
echo $TAMPERED
```

`git checkout -- src/index.js` removes the change **right away**. If you forget, your next commit will build and sign the tampered code as if it were legitimate.

`$TAMPERED` holds the full reference, such as `ghcr.io/koceeeneh/docutrust@sha256:cd884268...`

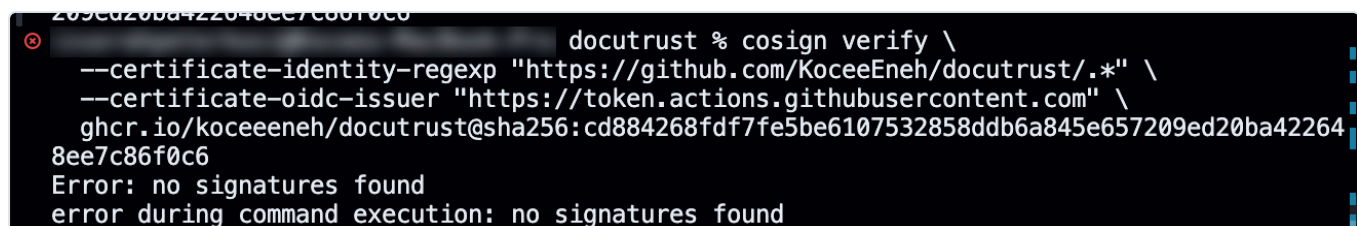
IF YOU SEE THIS

could not parse reference: ghcr.io/.../docutrust@ghcr.io/.../docutrust@sha256:... `docker inspect` already returns the full reference. Use `$TAMPERED` as it is, without putting `$IMAGE@` in front.

6.3 Try to verify it.

```
cosign2 verify --certificate-identity "$SIGNER" --certificate-oidc-issuer "$ISSUER" $TAMPERED
```

Expected: an error saying **no signatures were found**. The changed code has a new digest, and nothing ever signed that digest, however official the tag looks.



```
209ed20ba422648cc7c00f0c6
docutrust % cosign verify \
--certificate-identity-regexp "https://github.com/KoceeEneh/docutrust/.*" \
--certificate-oidc-issuer "https://token.actions.githubusercontent.com" \
ghcr.io/koceeeneh/docutrust@sha256:cd884268fdf7fe5be6107532858ddb6a845e657209ed20ba422648ee7c86f0c6
Error: no signatures found
error during command execution: no signatures found
```

Figure 10: the rejection message for the tampered digest.

Step 7: Identity forgery test (Deliverable 8)

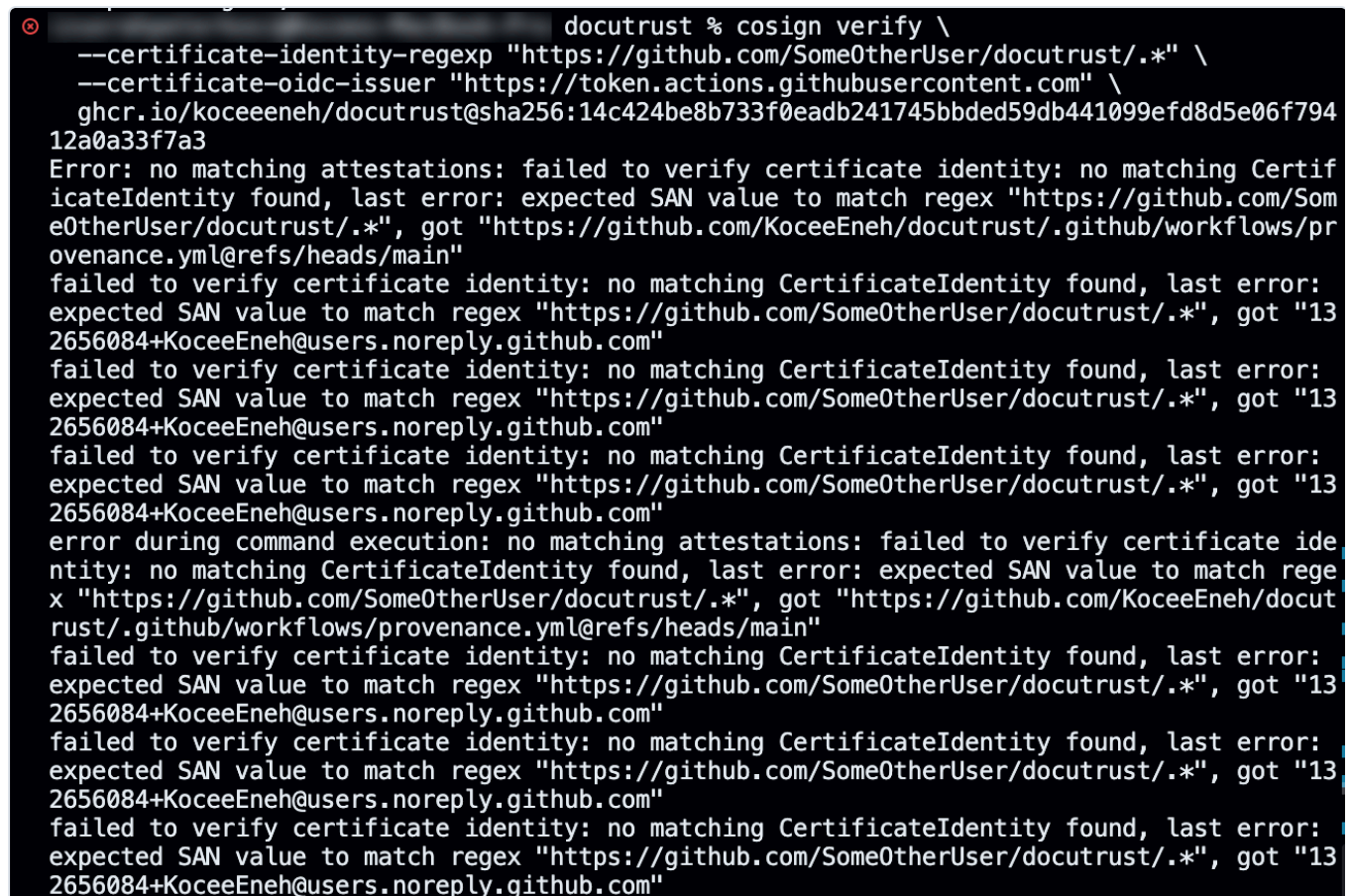
Could a signature from someone else's fork pass your check? Test the mirror image: verify your **real** signature against a policy that expects a different owner.

```
cosign2 verify \
--certificate-identity-regexp "https://github.com/SomeOtherUser/docutrust/.*" \
--certificate-oidc-issuer "$ISSUER" \
$IMAGE@$DIGEST
```

Expected: rejected. The error names the identity the certificate actually holds, for example

`https://github.com/KoceeEneh/docutrust/.github/workflows/provenance.yml@refs/heads/main`, and says it doesn't match the expected one. (The wording differs between Cosign versions; the rejection is what matters.)

A fork's workflow gets a certificate naming the **fork's** owner. GitHub sets that, not the workflow, so it can't pass a policy scoped to you.



```
docutrust % cosign verify \
  --certificate-identity-regexp "https://github.com/SomeOtherUser/docutrust/.*" \
  --certificate-oidc-issuer "https://token.actions.githubusercontent.com" \
  ghcr.io/koceeeneh/docutrust@sha256:14c424be8b733f0eadb241745bbded59db441099efd8d5e06f79412a0a33f7a3
Error: no matching attestations: failed to verify certificate identity: no matching CertificateIdentity found, last error: expected SAN value to match regex "https://github.com/SomeOtherUser/docutrust/.*", got "https://github.com/KoceeEneh/docutrust/.github/workflows/provenance.yml@refs/heads/main"
failed to verify certificate identity: no matching CertificateIdentity found, last error: expected SAN value to match regex "https://github.com/SomeOtherUser/docutrust/.*", got "132656084+KoceeEneh@users.noreply.github.com"
failed to verify certificate identity: no matching CertificateIdentity found, last error: expected SAN value to match regex "https://github.com/SomeOtherUser/docutrust/.*", got "132656084+KoceeEneh@users.noreply.github.com"
failed to verify certificate identity: no matching CertificateIdentity found, last error: expected SAN value to match regex "https://github.com/SomeOtherUser/docutrust/.*", got "132656084+KoceeEneh@users.noreply.github.com"
error during command execution: no matching attestations: failed to verify certificate identity: no matching CertificateIdentity found, last error: expected SAN value to match regex "https://github.com/SomeOtherUser/docutrust/.*", got "https://github.com/KoceeEneh/docutrust/.github/workflows/provenance.yml@refs/heads/main"
failed to verify certificate identity: no matching CertificateIdentity found, last error: expected SAN value to match regex "https://github.com/SomeOtherUser/docutrust/.*", got "132656084+KoceeEneh@users.noreply.github.com"
failed to verify certificate identity: no matching CertificateIdentity found, last error: expected SAN value to match regex "https://github.com/SomeOtherUser/docutrust/.*", got "132656084+KoceeEneh@users.noreply.github.com"
failed to verify certificate identity: no matching CertificateIdentity found, last error: expected SAN value to match regex "https://github.com/SomeOtherUser/docutrust/.*", got "132656084+KoceeEneh@users.noreply.github.com"
```

Figure 11: the rejection showing the expected and actual identity.

Step 8: Write the verification policy (Deliverable 9)

This is what Project 6 enforces. Write it with your own values:

```
Trusted identity: https://github.com/<you>/docutrust/.github/workflows/provenance.yml@refs/heads/main
Trusted issuer: https://token.actions.githubusercontent.com
```

Why:

- **Exact identity, not a regex.** It trusts one workflow file on `main`, and nothing else in the repo.
- **The workflow, not a person.** A person's account can sign anything, from anywhere. The workflow is repeatable and auditable.
- **Known limit:** any step in the job can obtain this identity (Project 4, Step 5).

Step 9: Handoff to Project 6 (Deliverable 10)

Record:

- The reference digest (`$DIGEST`) and what it carries: signature, SBOM attestation, provenance.
- The tampered digest (`$TAMPERED`). Project 6 reuses it.
- The identity and issuer from Step 8.
- The known gap carried over from Project 4, Step 5.

Part 3: Project 6: Admission-Time Enforcement and Attestation Graph Correlation

Goal: make the signatures matter at deploy time, then answer a real supply-chain question from the evidence. Kyverno will only admit images signed by your pipeline. GUAC will show whether the image you shipped contains the vulnerable lodash from Project 2.

Before you start, set your variables again in this terminal (Part 0.5 and Part 2):

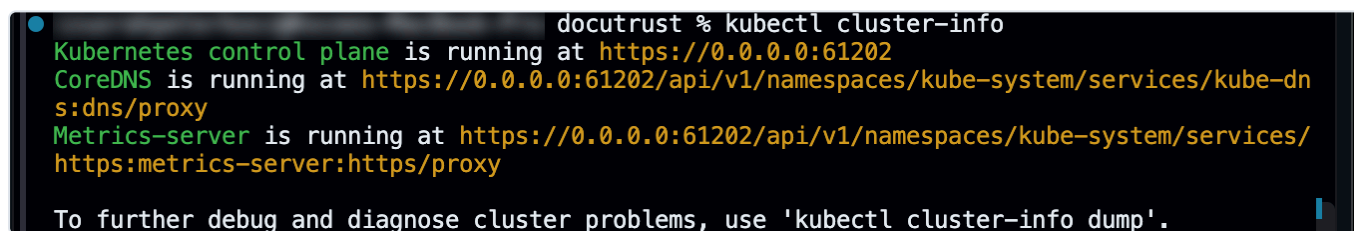
```
export GH_USER=<your-github-username>
export OWNER=$(echo "$GH_USER" | tr '[:upper:]' '[:lower:]')
export IMAGE=ghcr.io/$OWNER/docutrust
export DIGEST=<your digest from Part 2, Step 1.3>
export TAMPERED=<your tampered reference from Part 2, Step 6.2>
export SIGNER="https://github.com/$GH_USER/docutrust/.github/workflows/provenance.yml@refs/heads/main"
export ISSUER="https://token.actions.githubusercontent.com"
```

Step 1: Create a cluster and install Kyverno (Deliverable 1)

1.1 Create a local cluster.

```
k3d cluster create docutrust-admission-test
kubectl cluster-info
```

k3d runs a real Kubernetes cluster inside Docker containers.



```
docutrust % kubectl cluster-info
Kubernetes control plane is running at https://0.0.0.0:61202
CoreDNS is running at https://0.0.0.0:61202/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
Metrics-server is running at https://0.0.0.0:61202/api/v1/namespaces/kube-system/services/https:metrics-server:https/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

Figure 12: kubectl cluster-info against the new k3d cluster

1.2 Install Kyverno. The `maxContextSize` setting is needed in Step 6: a full SBOM is about 3 MB, and Kyverno's default limit is 2 MiB.

```
helm repo add kyverno https://kyverno.github.io/kyverno/
helm repo update
helm install kyverno kyverno/kyverno -n kyverno --create-namespace \
--set config.maxContextSize=8Mi
```

1.3 Wait until all pods are running.

```
kubectl get pods -n kyverno -w
```

Press **Ctrl+C** once all four show `1/1 Running` :

NAME	READY	STATUS	RESTARTS	AGE
kyverno-admission-controller-86cbbb5545-tp7m9	1/1	Running	0	3m46s
kyverno-background-controller-5546cb5b76-d62hh	1/1	Running	0	3m46s
kyverno-cleanup-controller-f947f9769-2dz7n	1/1	Running	0	3m46s
kyverno-reports-controller-79d68ccbb-dwvrj	1/1	Running	0	3m46s

IF YOU SEE THIS

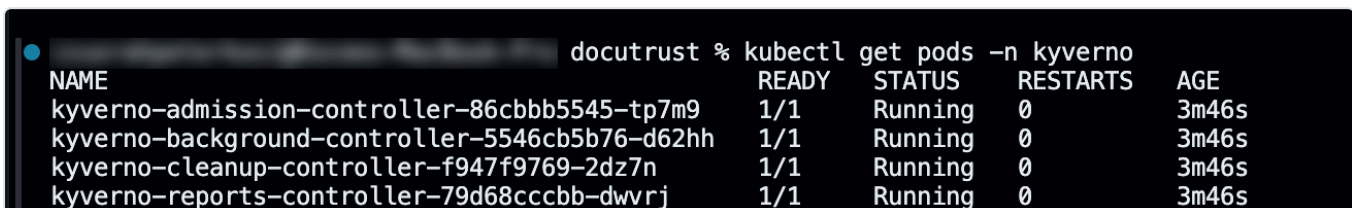
Pods stuck in `ImagePullBackOff`. Check the reason with `kubectl describe pod -n kyverno -l app.kubernetes.io/component=admission-controller | tail -20`. `lookup ghcr.io: Try again` is a temporary DNS failure inside the cluster. Kubernetes retries by itself; wait two minutes.

IF YOU SEE THIS

Startup probe failed: `... tls: internal error`. Kyverno is still creating its certificates on first start. Wait a minute.

NOTE

Kyverno runs with `enableTuf=false` by default. It checks signatures against the Sigstore root certificates built into this Kyverno release, and never downloads updated ones. That's fine for a lab. In production, turn on TUF or pin a trusted root.



NAME	READY	STATUS	RESTARTS	AGE
kyverno-admission-controller-86cbbb5545-tp7m9	1/1	Running	0	3m46s
kyverno-background-controller-5546cb5b76-d62hh	1/1	Running	0	3m46s
kyverno-cleanup-controller-f947f9769-2dz7n	1/1	Running	0	3m46s
kyverno-reports-controller-79d68cccbb-dwvrj	1/1	Running	0	3m46s

Figure 13: all four Kyverno pods Running.

Step 2: Signature policy (Deliverable 2)

Keep the Kubernetes files in the repo, so others can reuse them:

```
mkdir -p k8s/project6
```

```
cat > k8s/project6/kyverno-verify-docutrust-signature.yaml << EOF
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: verify-docutrust-signature
spec:
  validationFailureAction: Enforce
  webhookTimeoutSeconds: 30
  rules:
    - name: verify-docutrust-image-signature
      match:
        any:
          - resources:
              kinds:
                - Pod
      verifyImages:
        - imageReferences:
            - "$IMAGE*"
          attestors:
            - entries:
                - keyless:
                    subject: "$SIGNER"
                    issuer: "$ISSUER"
                    rekor:
                      url: https://rekor.sigstore.dev
```

EOF

```
cat k8s/project6/kyverno-verify-docutrust-signature.yaml
kubectl apply -f k8s/project6/kyverno-verify-docutrust-signature.yaml
kubectl get clusterpolicy verify-docutrust-signature
```

Check the `cat` output: `imageReferences`, `subject` and `issuer` must show your real values, not `$IMAGE`.

NAME	ADMISSION	BACKGROUND	READY	AGE	MESSAGE
verify-docutrust-signature	true	true	True	39s	Ready

Field	Meaning
<code>validationFailureAction: Enforce</code>	Blocks pods that fail. <code>Audit</code> would only log them.
<code>imageReferences</code>	Applies only to your DocuTrust images.
<code>subject / issuer</code>	Your exact values from Project 5, Step 8.
<code>rekor.url</code>	The signature must also be in the public log.

NOTE

Warning: `kyverno.io/v1 ClusterPolicy` is deprecated. The policy still works on Kyverno 1.19. The replacement API is `ImageValidatingPolicy`.

CAPTURE the policy showing `READY True`.

Step 3: Positive test: the signed image is admitted (Deliverable 3)

```
cat > k8s/project6/pod-signed-docutrust.yaml << EOF
apiVersion: v1
kind: Pod
metadata:
  name: docutrust-signed-test
spec:
  containers:
    - name: docutrust
      image: $IMAGE@$DIGEST
EOF
kubectl apply -f k8s/project6/pod-signed-docutrust.yaml
kubectl get pod docutrust-signed-test
```

```
pod/docutrust-signed-test created
NAME                READY   STATUS    RESTARTS   AGE
docutrust-signed-test 1/1     Running   0           3m33s
```

`created` means Kyverno checked the signature and let the pod in.

IF YOU SEE THIS

The pod is created but shows `ErrImagePull`. The package is still private (Part 1, Step 2.5). Admission has already passed at this point; this is only about pulling the image.

```

docutrust % kubectl get pod docutrust-signed-test
NAME          READY   STATUS    RESTARTS   AGE
docutrust-signed-test 1/1     Running   0           3m33s
docutrust %

```

Figure 14: pod/docutrust-signed-test created and the pod Running.

Step 4: Negative test: an unsigned image is rejected (Deliverable 4)

The test image has to match `$IMAGE*`, or the policy never looks at it. Build and push one without signing it. This uses your `docker login` from Project 5, Step 6.1.

```

docker build -t $IMAGE:unsigned-test .
docker push $IMAGE:unsigned-test
export UNSIGNED=$(docker inspect --format='{{index .RepoDigests 0}}' $IMAGE:unsigned-test)

cat > k8s/project6/pod-unsigned-test.yaml << EOF
apiVersion: v1
kind: Pod
metadata:
  name: docutrust-unsigned-test
spec:
  containers:
    - name: docutrust
      image: $UNSIGNED
EOF
kubectl apply -f k8s/project6/pod-unsigned-test.yaml

```

```

dc196d8/c82d: Layer already exists
385941412296: Layer already exists
unsigned-test: digest: sha256:9cd42624b5c9d006892bc9d09b2e379e9282ecad1ee58f65dd3664a8f10539da size: 856

```

Figure 15: The unsigned image pushed to GHCR, with the digest to deploy

```

Error from server: error when creating "pod-unsigned-test.yaml": admission webhook "mutate.kyverno.svc-fail" denied the request:

resource Pod/default/docutrust-unsigned-test was blocked due to the following policies

verify-docutrust-signature:
  verify-docutrust-image-signature: 'failed to verify image ghcr.io/koceeeneh/docutrust@sha256:9cd42624b5c9d006892bc9d09b2e379e9282ecad1ee58f65dd3664a8f10539da size: 856: no signatures found'

```

The pod was never created.

```

docutrust % cat > pod-unsigned-test.yaml << 'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: docutrust-unsigned-test
spec:
  containers:
    - name: docutrust
      image: ghcr.io/koceeeneh/docutrust@sha256:9cd42624b5c9d006892bc9d09b2e379e9282ecad1ee58f65dd3664a8f10539da
EOF

kubectl apply -f pod-unsigned-test.yaml
Error from server: error when creating "pod-unsigned-test.yaml": admission webhook "mutate.kyverno.svc-fail" denied the request:

resource Pod/default/docutrust-unsigned-test was blocked due to the following policies

verify-docutrust-signature:
  verify-docutrust-image-signature: 'failed to verify image ghcr.io/koceeeneh/docutrust@sha256:9cd42624b5c9d006892bc9d09b2e379e9282ecad1ee58f65dd3664a8f10539da: .attestors[0].entries[0].keyless: no signatures found'

```

Figure 16: the unsigned image being blocked.

Step 5: Negative test: the tampered image is rejected (Deliverable 5)

Reuse the tampered image from Project 5:

```

cat > k8s/project6/pod-tampered-test.yaml << EOF
apiVersion: v1
kind: Pod
metadata:
  name: docutrust-tampered-test
spec:
  containers:
    - name: docutrust
      image: $TAMPERED
EOF

kubectl apply -f k8s/project6/pod-tampered-test.yaml

```

Expected: blocked by `verify-docutrust-signature` with `no signatures found`, the same way as Step 4. The changed code has a digest the pipeline never signed.

```

docutrust % cat > pod-tampered-test.yaml << 'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: docutrust-tampered-test
spec:
  containers:
    - name: docutrust
      image: ghcr.io/koceeeneh/docutrust@sha256:cd884268fdf7fe5be6107532858ddb6a845e657209ed20ba422648ee7c86f0c6
EOF

kubectl apply -f pod-tampered-test.yaml
Error from server: error when creating "pod-tampered-test.yaml": admission webhook "mutate.kyverno.svc-fail" denied the request:

resource Pod/default/docutrust-tampered-test was blocked due to the following policies

verify-docutrust-signature:
  verify-docutrust-image-signature: 'failed to verify image ghcr.io/koceeeneh/docutrust@sha256:cd884268fdf7fe5be6107532858ddb6a845e657209ed20ba422648ee7c86f0c6: .attestors[0].entries[0].keyless: no signatures found'
```

Figure 17: the tampered image being blocked.

Step 6: SBOM attestation policy (Deliverable 6)

A signature alone doesn't say what's inside the image. This policy also requires a **pipeline-signed SBOM** that lists packages.

6.1 Create and apply the policy.

```

cat > k8s/project6/kyverno-verify-docutrust-sbom.yaml << EOF
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: verify-docutrust-sbom-attestation
spec:
  validationFailureAction: Enforce
  webhookTimeoutSeconds: 30
  rules:
    - name: verify-docutrust-sbom
      match:
        any:
          - resources:
              kinds:
                - Pod
      verifyImages:
        - imageReferences:
            - "$IMAGE*"
          attestations:
            - type: https://spdx.dev/Document
              attestors:
                - entries:
                    - keyless:
                        subject: "$SIGNER"
                        issuer: "$ISSUER"
                        rekor:
                            url: https://rekor.sigstore.dev
          conditions:
            - all:
                - key: "{{ length(packages) }}"
                  operator: GreaterThan
                  value: 0
```

EOF

```
kubectl apply -f k8s/project6/kyverno-verify-docutrust-sbom.yaml
```

`conditions` is checked against the **decoded SBOM**, so `packages` is Syft's real package list. An attestation that is validly signed but empty would still be rejected.

6.2 Positive test. Re-create the signed pod. It now has to pass **both** policies:

```
kubectl delete pod docutrust-signed-test --ignore-not-found
kubectl apply -f k8s/project6/pod-signed-docutrust.yaml
kubectl get pod docutrust-signed-test -o jsonpath='{.metadata.annotations.kyverno\.io/verify-images}';
echo
```

```
pod/docutrust-signed-test created
{"ghcr.io/koceeneh/docutrust@sha256:c9167a9d...": "pass"}
```

The annotation is Kyverno's own record that the image passed.

IF YOU SEE THIS

context size limit exceeded: 3162975 bytes exceeds limit of 2097152 bytes . You installed Kyverno without `config.maxContextSize` . Fix it with `helm upgrade kyverno kyverno/kyverno -n kyverno --reuse-values --set config.maxContextSize=8Mi` . It's a ConfigMap setting, not a startup flag: `admissionController.container.extraArgs.maxContextSize` crash-loops the admission controller. Also don't set it with `kubectl patch` and then `helm upgrade` , or Helm refuses with `conflict with "kubectl-patch"` .

IF YOU SEE THIS

`no matching attestations:` with nothing after the colon. The SBOM was attested with Cosign **v3** from a laptop. v3 stores it in a newer format that this Kyverno policy doesn't read. Attest in CI with the pinned v2.5.2, as in Project 5, Step 1.

```
docutrust % kubectl apply -f pod-signed-docutrust.yaml
Error from server: error when creating "pod-signed-docutrust.yaml": admission webhook "mutate.kyverno.svc-fail" denied the request:

resource Pod/default/docutrust-signed-test was blocked due to the following policies
verify-docutrust-sbom-attestation:
  verify-docutrust-sbom: 'image attestations verification failed, verifiedCount: 0, requiredCount: 1, error: no matching attestations: '
docutrust % kubectl apply -f pod-unsigned-test.yaml
Error from server: error when creating "pod-unsigned-test.yaml": admission webhook "mutate.kyverno.svc-fail" denied the request:

resource Pod/default/docutrust-unsigned-test was blocked due to the following policies
verify-docutrust-sbom-attestation:
  verify-docutrust-sbom: 'image attestations verification failed, verifiedCount: 0, requiredCount: 1, error: no matching attestations: '
verify-docutrust-signature:
  verify-docutrust-image-signature: 'failed to verify image ghcr.io/koceeneh/docutrust@sha256:9cd42624b5c9d006892bc9d09b2e379e9282ecad1ee58f65dd3664a8f10539da: .attestors[0].entries[0].keyless: no signatures found'
```

Figure 18: What that looks like: the SBOM policy rejecting an image whose SBOM was attested with Cosign v3

6.3 Identity test. Prove the policy pins the **exact workflow file**. Apply a temporary policy that trusts a different workflow in the same repo (`ci.yml`):

```
sed "s#provenance.yml#ci.yml#; s#verify-docutrust-sbom-attestation#test-wrong-workflow-identity#; s#name: verify-docutrust-sbom\$#name: expect-ci-yml-identity#" \
  k8s/project6/kyverno-verify-docutrust-sbom.yaml > k8s/project6/kyverno-identity-negative-test.yaml
grep -E "name:|subject:" k8s/project6/kyverno-identity-negative-test.yaml
kubectl apply -f k8s/project6/kyverno-identity-negative-test.yaml

cat > k8s/project6/pod-identity-test.yaml << EOF
apiVersion: v1
kind: Pod
metadata:
  name: docutrust-identity-test
spec:
  containers:
    - name: docutrust
      image: $IMAGE@$DIGEST
EOF
kubectl apply -f k8s/project6/pod-identity-test.yaml
```

`grep` should show the new policy name and a `subject` ending in `ci.yml@refs/heads/main`.

```
test-wrong-workflow-identity:
  expect-ci-yml-identity: 'image attestations verification failed, verifiedCount: 0, requiredCount: 1,
  error: subject mismatch: expected https://github.com/KoceeEneh/docutrust/.github/workflows/ci.yml@refs/heads/main,
  received https://github.com/KoceeEneh/docutrust/.github/workflows/provenance.yml@refs/heads/main'
```

Only the test policy blocked it. The real SBOM policy passed the same image in 6.2. Remove the test policy:

```
kubectl delete clusterpolicy test-wrong-workflow-identity
```

CAPTURE the "pass" annotation from 6.2.

CAPTURE the subject mismatch rejection from 6.3.

Step 7: Install GUAC and load the evidence (Deliverable 7)

GUAC stores SBOMs, provenance and vulnerability data in one graph you can query. Use a separate folder, **outside** the repo.

WINDOWS

In WSL, work in your Linux home folder (`~`), not under `/mnt/c/` . Some files below have a `:` in their name, which Windows folders don't allow.

7.1 Download GUAC v1.1.0.

```
mkdir -p ~/guac-lab && cd ~/guac-lab
gh release download v1.1.0 --repo guacsec/guac --pattern 'guac-demo-compose.yaml'
```

macOS:

```
gh release download v1.1.0 --repo guacsec/guac --pattern 'guacone-darwin-amd64'
mv guacone-darwin-amd64 guacone && chmod +x guacone
```

```
xattr -d com.apple.quarantine guacone 2>/dev/null
file guacone
```

`file` shows `Mach-O universal binary with 2 architectures: [x86_64] [arm64]`. Despite the `amd64` name, it runs natively on Apple Silicon. You don't need Rosetta.

WINDOWS

In WSL, download `guacone-linux-amd64` instead:

```
gh release download v1.1.0 --repo guacsec/guac --pattern 'guacone-linux-amd64'
mv guacone-linux-amd64 guacone && chmod +x guacone
```

7.2 Start GUAC in a second terminal and leave it running:

```
cd ~/guac-lab && docker compose -f guac-demo-compose.yaml -p guac up --force-recreate
```

Back in your first terminal, check it's up:

```
docker compose -p guac ps --format '{{.Service}}'
```

You should see `graphql`, `collectsub`, `depsdev-collector`, `osv-certifier` and others. The GraphQL API is at `http://localhost:8080/query`.

7.3 Load the SBOM, taken from the verified attestation. Only load what you've verified:

```
cd ~/guac-lab && mkdir -p sbom-verified
cosign2 verify-attestation --type spdxjson \
  --certificate-identity "$SIGNER" --certificate-oidc-issuer "$ISSUER" \
  $IMAGE@$DIGEST > verified-att.jsonl
head -1 verified-att.jsonl | jq -r '.payload' | base64 -d | jq '.predicate' > sbom-verified/docutrust-sbom.spdx.json
jq '.packages | length' sbom-verified/docutrust-sbom.spdx.json

./guacone collect --add-vuln-on-ingest files sbom-verified
```

`--add-vuln-on-ingest` looks up each package in the OSV vulnerability database while loading. Look for these lines in the output:

```
assembling Vulnerability: 75
assembling CertifyVuln: 389
assembling HasSBOM: 1
completed ingesting 1 documents of 1
```

7.4 Load the provenance.

```
mkdir -p provenance-only
gh attestation download oci://$IMAGE@$DIGEST --owner $GH_USER
head -1 sha256*.jsonl | jq -r '.dsseEnvelope.payload' | base64 -d > provenance-only/docutrust-provenance.intoto.json
jq '{predicateType, subject}' provenance-only/docutrust-provenance.intoto.json

./guacone collect files provenance-only
```

Expected: `predicateType` is `https://slsa.dev/provenance/v1`, the subject digest matches yours, and the load output shows `assembling HasSLSA: 1`.

NOTE

What this costs. You load the provenance **without** its signature envelope, so GUAC stores it without verifying it. Trust in this record comes from the checks you ran earlier: `gh attestation verify` and Kyverno. Say this in Deliverable 9.

IF YOU SEE THIS

`failed to verify identity: failed to find key from key providers`. You loaded the signed envelope itself. GUAC v1.1.0 can only verify signatures from keys it's configured with, and this signature is keyless. Load the decoded statement instead, as above.

IF YOU SEE THIS

`no document parser registered for type: ITE6`. You loaded the SBOM **attestation statement**. GUAC v1.1.0 can't read an SBOM wrapped in an in-toto statement. Load the plain SPDX file (the `.predicate`), as in 7.3.

```
guac-lab % docker compose ls --filter "name=guac"
NAME                STATUS              CONFIG FILES
guac                 running(6)          e.yaml
```

Figure 19: docker compose ps showing GUAC running.

CAPTURE the ingest output with HasSBOM: 1 and Vulnerability: 75.

Step 8: Answer the Iodash question from the graph (Deliverable 8)

The question: does the image your pipeline built, signed and admitted contain the vulnerable Iodash that Project 2 found?

8.1 A query helper. This saves writing `curl` in full every time:

```
gql() { jq -n --arg q "$1" '{query: $q}' | curl -s http://localhost:8080/query \
  -H 'content-type: application/json' --data @- ; }
export D=${DIGEST#sha256:} # the digest without the "sha256:" prefix, as GUAC stores it
```

8.2 Link the image to its SBOM. The SBOM and the provenance are stored under **different** digests:

- The provenance uses the registry digest (`$DIGEST`), like Cosign and Kyverno.
- Syft records the image under a digest it computes itself, and GUAC uses that one.

```
export SYFT_DIGEST=$(jq -r '.packages[] | select(any(.externalRefs[]?; .referenceLocator | startswith
("pkg:oci")) | .checksums[0].checksumValue' sbom-verified/docutrust-sbom.spdx.json)
echo $SYFT_DIGEST
```

The signed statement you verified in 7.3 holds both digests: `$DIGEST` as its subject and `$SYFT_DIGEST` inside the SBOM. That's your evidence for recording a link between them:

```
gql "mutation { ingestHashEqual(artifact: {artifactInput: {algorithm: \"sha256\", digest: \"\$D\"}}, ot
herArtifact: {artifactInput: {algorithm: \"sha256\", digest: \"\$SYFT_DIGEST\"}}, hashEqual: {justifica
tion: \"Verified CI SBOM attestation: subject is the registry digest; SBOM records the Syft image dige
st\", origin: \"manual\", collector: \"manual\", documentRef: \"manual:chain-b-hashequal\"}) }" | jq
```

```
{ "data": { "ingestHashEqual": "71090" } }
```

IF YOU SEE THIS

IngestHashEqual :: Artifact not found . One of the digests isn't in GUAC yet. Finish 7.3 and 7.4 first.

NOTE

Syft's image digest changes between Syft versions: Syft 1.42.3 (CI) and 1.51.1 (local) gave the same image two different digests. That's why this guide takes the SBOM from the verified CI attestation, not from a fresh local scan.

8.3 Query the chain, one link at a time.

Built by:

```
gql "{ HasSLSA(hasSLSASpec: {subject: {digest: \"${D}\"}}) { slsa { builtBy { uri } } } }" | jq
```

```
"uri": "https://github.com/KoceeEneh/docutrust/.github/workflows/provenance.yml@refs/heads/main"
```

Same image as the SBOM subject:

```
gql "{ HashEqual(hashEqualSpec: {artifacts: [{digest: \"${D}\"}]} ) { artifacts { digest } justification } }" | jq
```

The SBOM contains lodash:

```
gql "{ HasSBOM(hasSBOMSpec: {subject: {artifact: {digest: \"${SYFT_DIGEST}\"}}) { origin includedSoftware { __typename ... on Package { namespaces { names { name versions { version } } } } } }" \
  | jq '.data.HasSBOM[] | {origin, lodash: [.includedSoftware[] | select(__typename=="Package") | .namespaces[].names[] | select(.name=="lodash") | .versions[].version}]'
```

```
"lodash": ["4.17.15"]
```

Known vulnerabilities in that lodash:

```
gql '{ CertifyVuln(certifyVulnSpec: {package: {type: "npm", name: "lodash"}}) { package { namespaces { names { versions { version } } } } vulnerability { vulnerabilityIDs { vulnerabilityID } } }' \
  | jq -r '.data.CertifyVuln[] | \"\\(.package.namespaces[0].names[0].versions[0].version) \\(.vulnerability.vulnerabilityIDs[0].vulnerabilityID)\"' | sort -u
```

```
4.17.15  ghsa-29mw-wpgm-hmr9
4.17.15  ghsa-35jh-r3h4-6jhm
4.17.15  ghsa-f23m-r3pf-42rh
4.17.15  ghsa-p6mc-m468-83gw
4.17.15  ghsa-r5fr-rjxr-66jc
4.17.15  ghsa-xxjr-mmjv-4gpg
```

8.4 Count distinct vulnerabilities, not IDs. Some advisories are aliases of each other. Check each one with OSV:

```
for id in $(gql '{ CertifyVuln(certifyVulnSpec: {package: {type: "npm", name: "lodash"}}) { vulnerability { vulnerabilityIDs { vulnerabilityID } } }' | jq -r '.data.CertifyVuln[].vulnerability.vulnerabilityIDs[0].vulnerabilityID' | sort -u); do
```

```
curl -s "https://api.osv.dev/v1/vulns/$(echo $id | sed 's/^ghsa-/GHSa-/' ) | jq -r '"(.id) | \((.aliases // []) | join(",")) | \(.summary)'"
done
```

In the reference build, the 6 IDs were **4 distinct vulnerabilities**:

Vulnerability	CVE	Advisory IDs
Prototype pollution	CVE-2020-8203	GHSa-p6mc-m468-83gw
Regular expression DoS	CVE-2020-28500	GHSa-29mw-wpgm-hmr9
Command/code injection via <code>_.template</code>	CVE-2021-23337	GHSa-35jh-r3h4-6jhm, GHSa-r5fr-rjxr-66jc
Prototype pollution in <code>_.unset</code> / <code>_.omit</code>	CVE-2025-13465	GHSa-f23m-r3pf-42rh, GHSa-xxjr-mmjv-4gpg

Answer: the image built by `provenance.yml` on `main`, signed, and admitted by Kyverno contains **lodash 4.17.15**, with **4 known vulnerabilities**.

NOTE

Graph counts are not SBOM counts. The `depsdev-collector` in GUAC's demo setup adds its own records for every package it sees (517 in the reference build). Always count packages from the SBOM file.

```
guac-lab % curl 'http://localhost:8080/query' -s -X POST -H 'content-type: application/json' \
--data '{"query":{"HasSLSA(hasSLSASpec: {}) { subject { algorithm digest } slsa { buildType builtBy { uri } slsaVersion } } } }' | jq
{
  "data": {
    "HasSLSA": [
      {
        "subject": {
          "algorithm": "sha256",
          "digest": "c9167a9d3425a8f19884ca5b696d228ce6afc65dcd688a18d4573f85eaf09017"
        },
        "slsa": {
          "buildType": "https://actions.github.io/buildtypes/workflow/v1",
          "builtBy": {
            "uri": "https://github.com/KoceeEneh/docutrust/.github/workflows/provenance.yml@refs/heads/main"
          },
          "slsaVersion": "https://slsa.dev/provenance/v1"
        }
      }
    ]
  }
}
```

Figure 20: the four chain queries and their results.

Step 9: Honest scope assessment of GUAC (Deliverable 9)

Answer these in your report, using your own results.

What GUAC added:

- Automatic vulnerability lookup during loading (75 vulnerabilities, 389 package links, from one SBOM).

- One place to query provenance, SBOM, vulnerabilities and identity links together.

What it didn't add at this scale:

- For one image, `jq` on the SBOM plus a query to OSV gives the same Iodash answer faster. GUAC's value is questions across **many** images over time ("which of our images contain Iodash 4.17.15?"), which a one-app lab can't show.

Limits you hit (GUAC v1.1.0):

Limit	Consequence
Can't verify keyless signatures on file loading	Provenance loaded unverified; trust comes from earlier checks
Can't read an SBOM inside an in-toto statement	SBOM loaded as a plain file, which loses the signed subject
Joins on Syft's own image digest	SBOM and provenance only connected through a manual <code>HashEqual</code>
Demo collectors add outside data	Graph counts $\neq$ SBOM counts

Verdict to write: in this setup GUAC connects evidence; it doesn't verify it. It is only as trustworthy as the checks run before loading, and only as useful as the identifiers the tools share.

Step 10: Chain B closing report (Deliverable 10)

Your report should have these five sections. Use the evidence from your screenshots.

1. **What was proven:** where the image came from (Project 4), that it's genuine and unchanged (Project 5), and that only proven images can run (Project 6).
2. **Trusted identity:** your `SIGNER` and `ISSUER` values.
3. **Reference digests:** the signed image (`$DIGEST`), the unsigned test (`$UNSIGNED`) and the tampered test (`$TAMPERED`), with the result for each.
4. **What was not proven:** that the image is safe. It passed every Chain B control and still contains Iodash 4.17.15 with 4 known vulnerabilities, plus the SQL injection from Project 4, Step 1.
5. **Open items:** signing isn't isolated from the build steps (Project 4, Step 5); Kyverno uses the deprecated `ClusterPolicy` API; `enableTuf=false`; the GUAC link is manual; the Source Track wasn't attempted.

Step 11: Commit your files and clean up

11.1 Keep scratch files out of Git, then commit the Kubernetes files.

```
cd <path-to>/docutrust
printf "\n# Chain B scratch files\nverified-att.json\nnsbom-verified.spx.json\n" >> .gitignore
git add .gitignore k8s/project6
git status --short
git commit -m "Project 6: Kyverno policies and admission test pods"
git push
```

`git status --short` should list only `.gitignore` and the files in `k8s/project6/`.

NOTE

Every push to `main` builds and signs a new image. That's expected: your evidence still points at `$DIGEST`.

11.2 Stop everything when you're done.

```
k3d cluster delete docutrust-admission-test  
cd ~/guac-lab && docker compose -p guac down
```

GITHUB WEBSITE

Delete the test images from GHCR under **Packages** → **docutrust** → **Package settings** → **Manage versions**. Delete the versions tagged `tampered-test` and `unsigned-test`.

Appendix A: The complete provenance.yml

This is the finished workflow after Parts 1 and 2, with the Project 4 test steps removed.

```

name: Build, Publish, and Attest Provenance

on:
  push:
    branches: [main]

permissions:
  contents: read
  packages: write
  id-token: write
  attestations: write

env:
  REGISTRY: ghcr.io

jobs:
  build-publish-attest:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Compute lowercase image name
        id: image
        run: |
          echo "name=$(echo '${{ github.repository }}' | tr '[:upper:]' '[:lower:]')" >> "$GITHUB_OUTP
UT"

      - name: Log in to GHCR
        uses: docker/login-action@v3
        with:
          registry: ${ env.REGISTRY }
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }

      - name: Build and push image
        id: push
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${ env.REGISTRY }/${ steps.image.outputs.name }:${ github.sha }

      - name: Generate build provenance attestation
        uses: actions/attest-build-provenance@v2
        with:
          subject-name: ${ env.REGISTRY }/${ steps.image.outputs.name }
          subject-digest: ${ steps.push.outputs.digest }
          push-to-registry: true

      - name: Install cosign
        uses: sigstore/cosign-installer@v3
        with:
          cosign-release: 'v2.5.2'

      - name: Sign the image with Cosign (keyless)
        run: |
          cosign sign --yes ${ env.REGISTRY }/${ steps.image.outputs.name }@${ steps.push.output
s.digest }

```

- **name:** Record cosign version
run: cosign version
- **name:** Install Syft
uses: anchore/sbom-action/download-syft@v0
with:
 syft-version: v1.42.3
- **name:** Generate SBOM from the pushed image
run: |
 syft \${env.REGISTRY}/\${steps.image.outputs.name}@\${steps.push.outputs.digest} \\
 -o spdx-json > sbom.json
 echo "Packages found: \$(jq '.packages | length' sbom.json)"
- **name:** Attest SBOM with Cosign (workflow identity)
run: |
 cosign attest --yes \\
 --predicate sbom.json \\
 --type spdxjson \\
 \${env.REGISTRY}/\${steps.image.outputs.name}@\${steps.push.outputs.digest}

Appendix B: Error index

Error message (short)	Where	Fix
repository name must be lowercase	P4 Step 2	Use the <code>Compute lowercase image name</code> step
Workflow does not have 'workflow_dispatch' trigger	P4 Step 5	Add <code>workflow_dispatch:</code> under <code>on:</code>
failed to get run: HTTP 404 for a run that exists	Any <code>gh run</code>	<code>gh</code> guessed the wrong repo. Run <code>gh repo set-default <you>/docutrust</code> , or add <code>--repo <you>/docutrust</code>
slsa-verifier: no matching attestations	P4 Step 7	Expected. Use <code>gh attestation verify</code>
parsing reference: ghcr.io/@	P5 Step 1	Cosign steps run before the build; move them to the end
The token provided does not match expected scopes	P5 Step 6	Classic token with <code>write:packages</code> ; <code>docker logout</code> then <code>docker login</code>
context deadline exceeded reaching ghcr.io	Any	Temporary network problem. Check with <code>curl -s -o /dev/null -w "%{http_code}\n" https://ghcr.io/v2/</code> (a 401 is fine), then retry
none of the attestations matched the predicate type	P5 Step 4, P6 Step 7	Use <code>cosign2</code> (v2.5.2), not v3
ImagePullBackOff / tls: internal error on Kyverno pods	P6 Step 1	Temporary at first start; wait
context size limit exceeded	P6 Step 6	<code>helm upgrade ... --set config.maxContextSize=8Mi</code>
conflict with "kubectl-patch" on helm upgrade	P6 Step 6	Don't mix <code>kubectl patch</code> and Helm on the same ConfigMap key
no matching attestations: (nothing after the colon)	P6 Step 6	SBOM was attested with Cosign v3; attest in CI with v2.5.2
x509: certificate signed by unknown authority from Kyverno	P6 Step 6	Seen only with SBOMs attested by local Cosign v3. Attest in CI

Error message (short)	Where	Fix
failed to find key from key providers	P6 Step 7	Load the decoded provenance statement, not the envelope
no document parser registered for type: ITE6	P6 Step 7	Load the plain SPDX file, not the attestation
Artifact not found on ingestHashEqual	P6 Step 8	Load both documents first
zsh parse error near '\n'	Anywhere	You pasted a <placeholder> . Replace it with your value

Appendix C: Versions used in the reference build

Component	Version
Machine	macOS, Apple Silicon
GitHub CLI	2.100.0
Cosign in CI, and cosign2 locally	v2.5.2
Cosign installed locally (cosign tree only)	v3.1.3
Syft in CI	v1.42.3
Syft local	v1.51.1
Kyverno	v1.19.1 (Helm chart 3.9.1)
GUAC	v1.1.0
GitHub Actions	checkout@v4 , docker/login-action@v3 , docker/build-push-action@v6 , attest-build-provenance@v2 , cosign-installer@v3 , anchore/sbom-action/download-syft@v0

Acknowledgements

Written, built and tested by **Kosisochukwu Eneh** for the Expadox Labs DevSecOps track.